SentinelDesk

Mini SOC Monitoring and Incident Response Platform

Project Overview & Feature Guide · Version 1.1 · October 2026

At a glance

SentinelDesk is a small, self-contained Security Operations Center (SOC) application. It ingests Windows-style security logs, runs detection rules, raises alerts with supporting evidence, and gives an analyst the tools to investigate alerts and manage incidents from first report to resolution and export. It is a portfolio project for SOC Analyst L1 roles, designed to be simple to run, test and explain.

Purpose	Simulate the daily L1 SOC workflow: monitor → detect → triage → escalate → report
Stack	React + Vite + Tailwind CSS + Recharts · Python FastAPI · SQLAlchemy · SQLite · pytest
Log sources	JSON and CSV security logs · Windows Event IDs 4624, 4625, 4720, 7045
Detections	4 tunable rules mapped to MITRE ATT&CK, each with documented limitations and false positives
Workflow	Alert verdicts (Suspected → Confirmed with justification) · Incidents (New → Investigating → Escalated → Resolved)
Outputs	Live dashboard · incident timeline · Markdown and JSON incident reports
Cost	Free and offline: no paid services, no cloud accounts, synthetic data only

Build status: complete. All 8 build phases are finished. **149 automated tests pass**, and a browser end-to-end test runs the full analyst workflow on an empty database with zero console errors: upload → alert → note → justified Confirmed verdict → incident → Investigating → Escalated → Resolved → report export → dashboard update.

1. Project overview

Why this project exists

An L1 SOC analyst spends most of the day on a repeating loop: watch the alert queue, look at the raw events behind each alert, decide whether the activity is malicious or benign, write notes, and escalate real problems as incidents. Commercial SIEM tools (Splunk, Microsoft Sentinel, QRadar) hide much of that logic. SentinelDesk rebuilds the core of it in a small codebase to show *how* detections work, *why* they produce false positives, and *how* an investigation is recorded.

Goals

- Show practical knowledge of Windows security events and common attack patterns (brute force, persistence, account creation).
- Show sound triage judgement: a rule match is a *signal*, not proof. Nothing is called a confirmed attack without written evidence.
- Show secure development habits: input validation, parameterized queries, output escaping, secrets in environment variables, no execution of uploaded content.
- Show engineering discipline: separate detection logic, configurable thresholds, automated tests and clear documentation.

Who uses it (the simulated roles)

Role	What they do in SentinelDesk
SOC Analyst (L1)	Uploads logs, works the alert queue, reviews evidence and related events, adds notes, sets verdicts, opens incidents.
Escalation (L2 / IR)	Receives incidents moved to **Escalated**, records the response, and resolves with a summary.
Detection engineer	Tunes rule thresholds and windows and reviews each rule's limitations and false positives.

Technology stack

Layer	Technology	Why it was chosen
Frontend	React 18, Vite 5, React Router 6	Component-based UI with fast reload; one page per SOC task.
Styling	Tailwind CSS 3	Consistent dark SOC theme without large CSS files.
Charts	Recharts 2	Simple React charts; every chart also has a table view.
Backend	Python 3.12+, FastAPI, Pydantic 2	Input validation and API docs (/docs) generated automatically.
Database	SQLite + SQLAlchemy 2	A single file with nothing to install; the ORM uses parameterized queries.
Config	pydantic-settings, .env	Secrets and limits come from environment variables.
Testing	pytest, FastAPI TestClient, Playwright	149 automated tests, plus a browser end-to-end check during development.

2. Architecture

SentinelDesk uses a three-tier design: a browser frontend, a REST API backend and a database. The key design decision is that **detection logic lives in its own package** (`app/detection/`) with no web or database code. Each rule receives plain event records plus its settings and returns alert results, so rules are easy to unit-test with known suspicious and normal data.

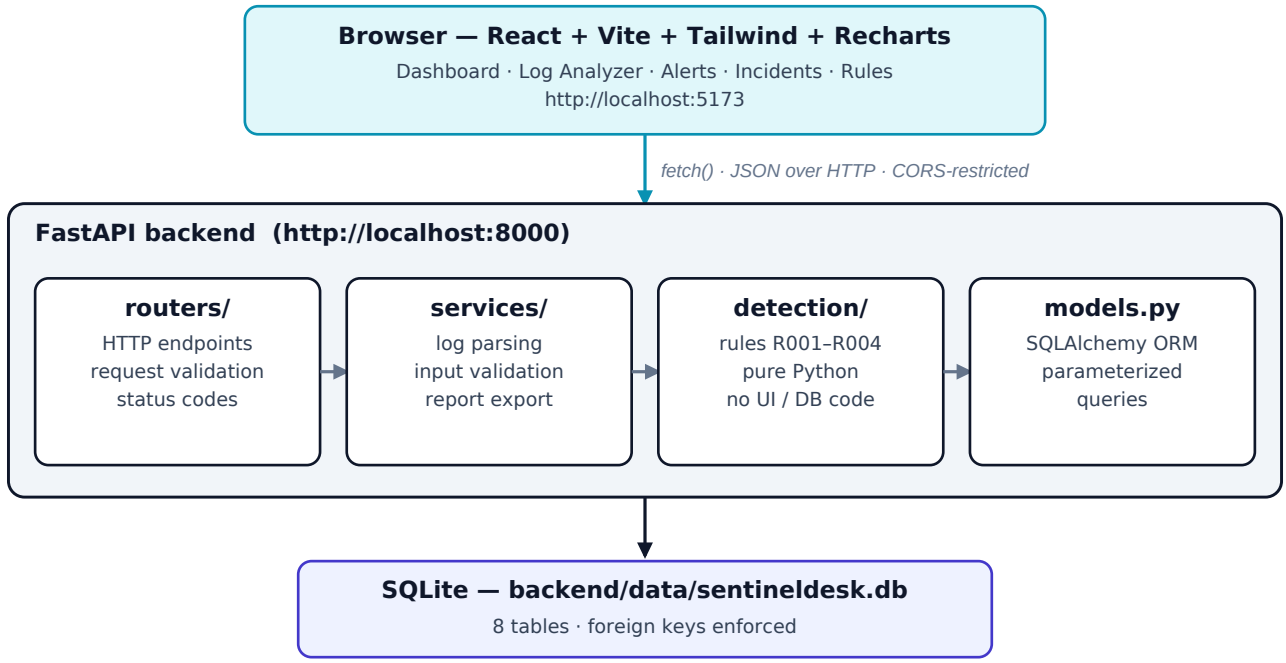


Figure 1 — Component architecture

End-to-end data flow



Figure 2 — From raw log to incident report

Folder / file	Responsibility
`app/main.py`	Starts the API, creates tables, seeds rules, configures CORS and safe error handling.
`app/routers/`	HTTP endpoints only: health, logs, events, rules, alerts, incidents, dashboard.
`app/services/log_parser.py`	File and row validation, normalization, field-name aliases, UTC conversion.
`app/services/ingest_service.py`	Saves an upload and its events, then runs detection (shared by the API and the demo loader).
`app/services/detection_service.py`	Loads rule settings, runs the engine and saves alerts as Suspected with evidence links.
`app/services/report_service.py`	Builds incident reports as JSON or escaped Markdown.
`app/detection/`	Event catalog, engine and one file per rule (R001–R004).
`app/demo_data.py`	One-command demo loader: python -m app.demo_data --reset.
`frontend/src/pages/`	Dashboard, Log Analyzer, Alerts, Alert detail, Incidents, New incident, Incident detail, Rules.

3. Features

3.1 SOC Dashboard

Status: Built and tested

- **Stat cards:** Total Alerts, Critical Alerts, Open Incidents (New / Investigating / Escalated), Resolved Incidents. Each card links to the matching filtered list.
- **Verdict strip:** counts of Suspected, Confirmed, False Positive and Benign alerts.
- **Charts:** alerts by severity (status colors checked for colour-blind separation and contrast) and alerts by event category. Hover tooltips, plus a **Table** toggle that shows the exact numbers.
- **Recent security events** and **latest alerts**. The page refreshes every 30 seconds and shows a guided empty state when there is no data.
- **Dark / light mode:** a header toggle. The first visit follows the operating-system setting and the choice is remembered. Light mode mirrors every color scale through CSS variables, and the charts switch to colors validated for each background.

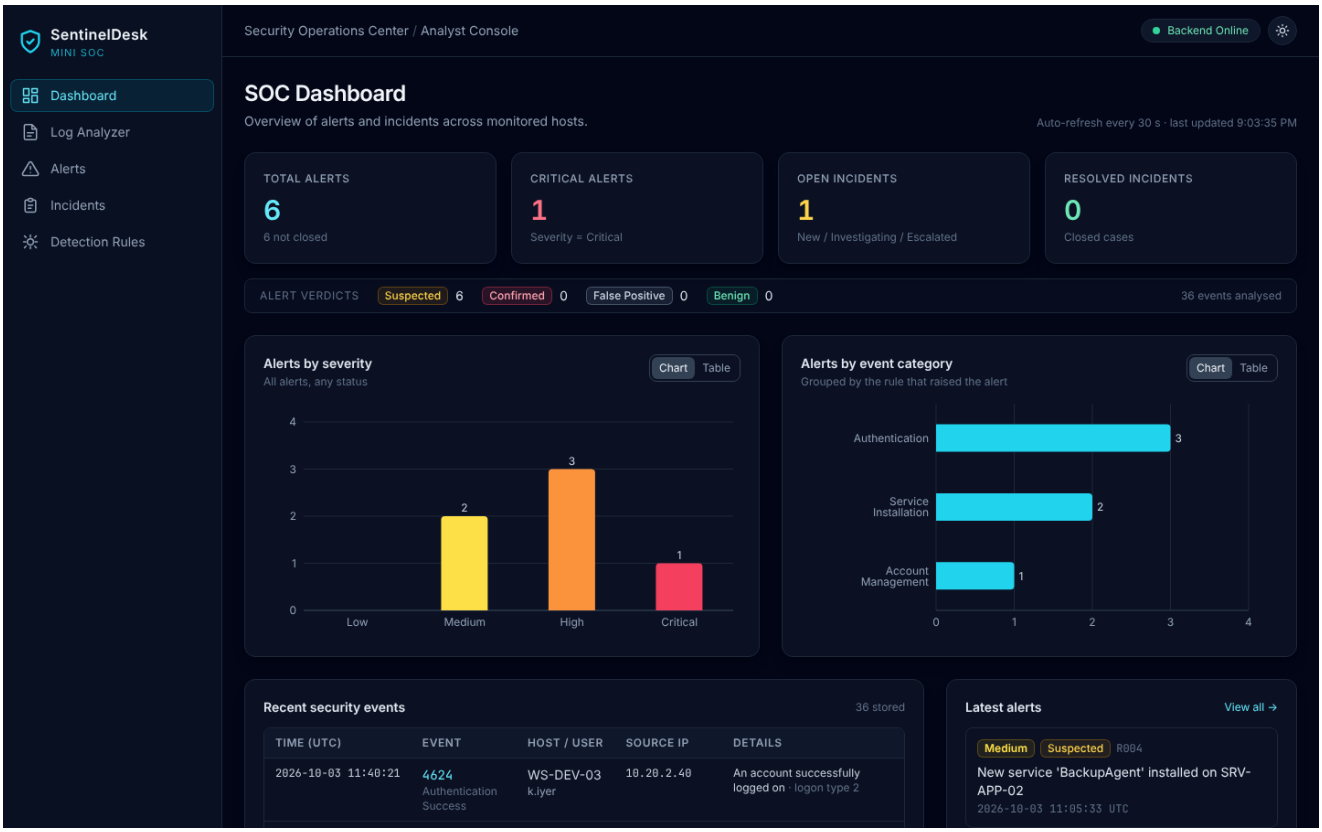


Figure 3 — SOC dashboard with the demo data loaded (dark mode)

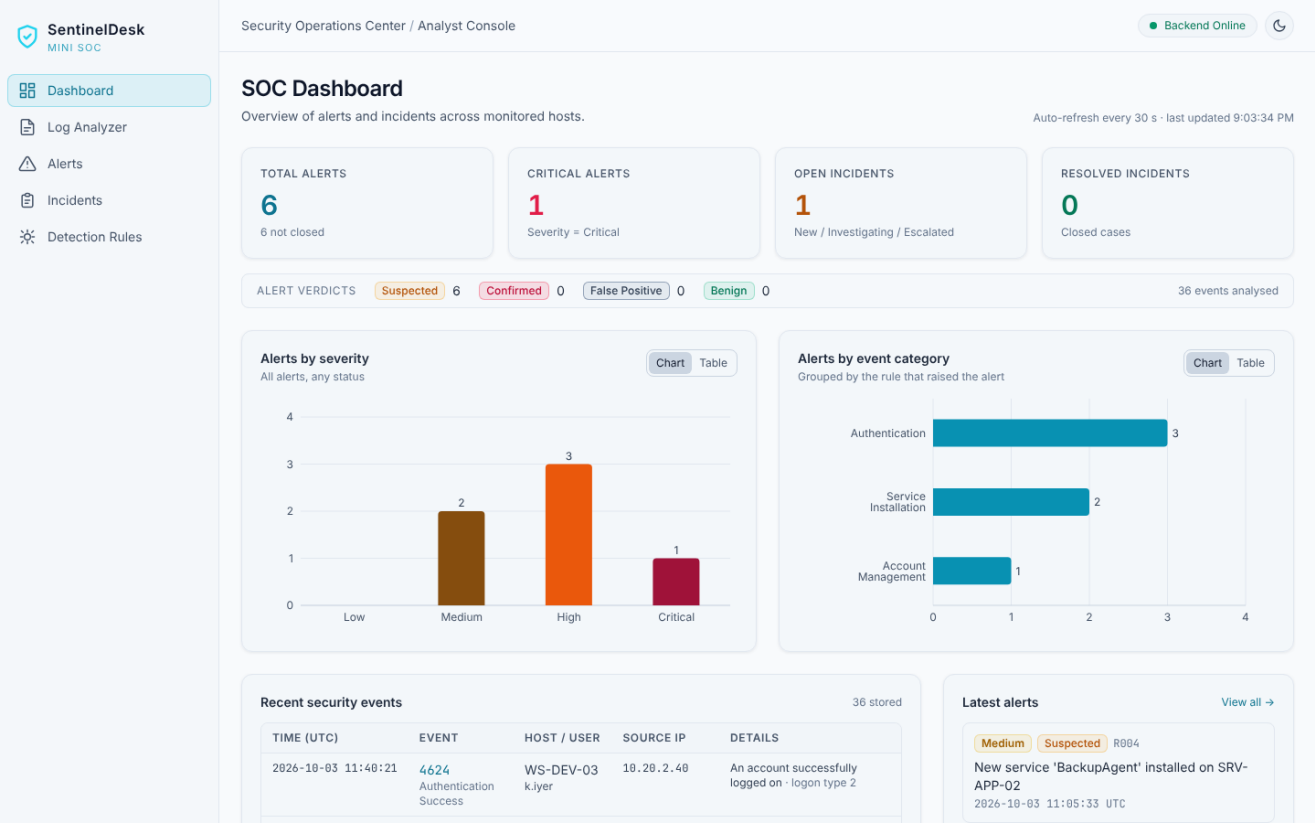


Figure 3b — The same dashboard in light mode

3.2 Security Log Analyzer

Status: Built and tested

Upload by drag and drop or the file picker. Files are read into memory and parsed only with Python's `json` and `csv` modules. They are never saved as files and never executed. Valid events are stored with a category from the Event ID catalog and the original row (`raw_data`), and the detection rules run automatically. The result shows accepted and rejected counts, each rejected row with its reason, and links to the generated alerts.

Validation

- Only `.json` / `.csv`; maximum 2 MB and 10,000 events (set by environment variables). Only the limit + 1 byte is read.
- Binary content, invalid UTF-8, broken JSON, and CSV files without timestamp / event_id columns are rejected with a clear message.
- Each row is checked on its own: timestamp (ISO 8601, converted to UTC), Event ID (1–65535), IP address format, plain-text values and field lengths.
- Alternative column names (EventID, TimeCreated, Computer, IpAddress) and the BOM that Excel adds to CSV files are handled. Filenames are reduced to a safe base name.

Situation	HTTP	Result
Valid file (some rows may be invalid)	201	Valid rows stored and checked by rules; bad rows listed by number and reason
No valid rows	422	Nothing saved; row errors returned
Wrong extension	415	“Only .json and .csv files are accepted”
Over size or event limit	413	Rejected

Situation	HTTP	Result
Empty, binary, non-UTF-8, broken JSON/CSV	400	Rejected with a readable message

Windows Event IDs analysed

Event ID	Meaning	Category	Why an analyst cares
4624	Successful logon	Authentication Success	Suspicious right after many failures or from an unusual source
4625	Failed logon	Authentication Failure	Many failures in a short time can indicate password guessing
4720	User account created	Account Management	Attackers create accounts to keep access (persistence)
7045	Service installed	Service Installation	Services run automatically, often as SYSTEM, a common persistence method

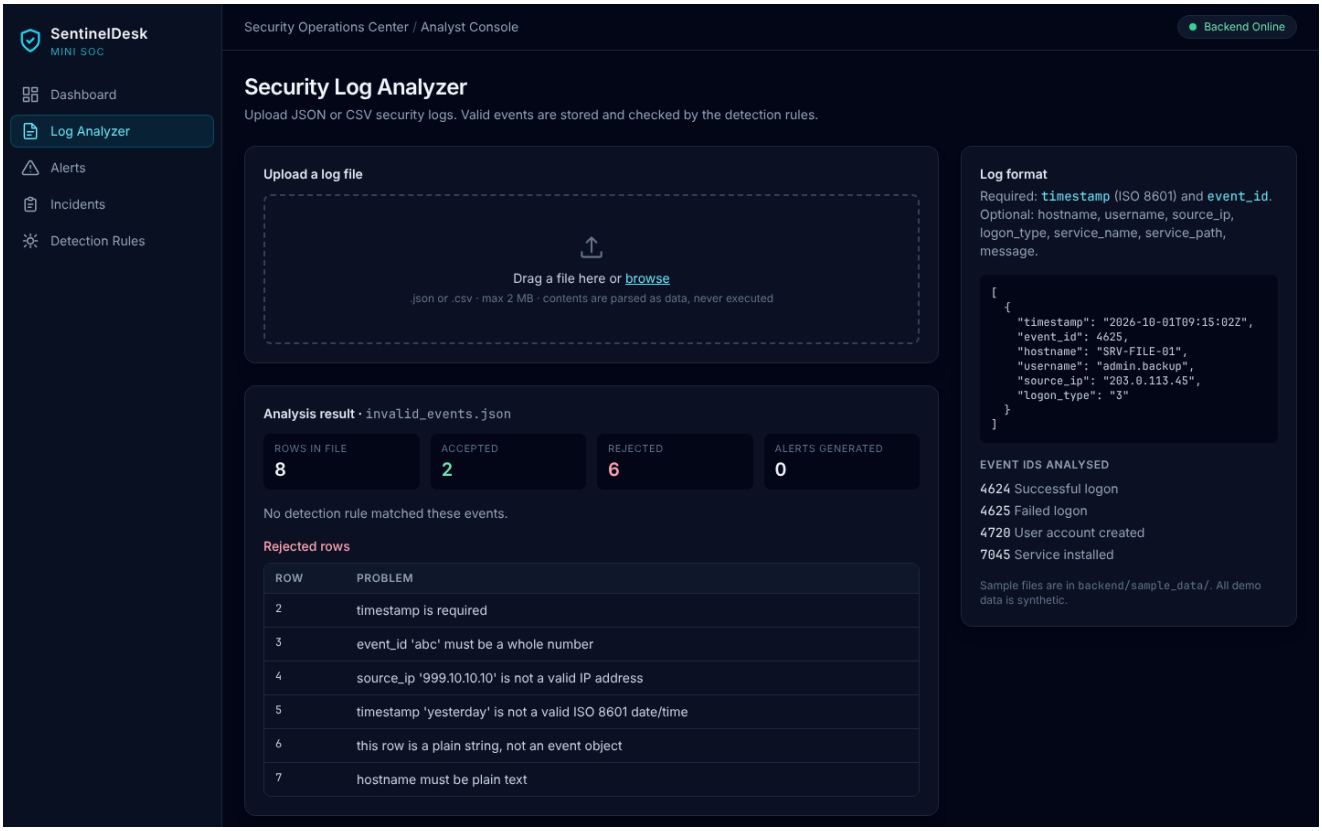


Figure 4 — Uploading invalid_events.json: 2 rows accepted, 6 rejected with reasons

3.3 Alert Investigation

Status: Built and tested

- **Alert queue** with search (title, host, user, IP) and filters for severity, status, verdict and rule. Filters are kept in the URL, so views can be bookmarked.
- **Why this alert fired:** a plain-English explanation with counts, times and the threshold used, plus key evidence values.
- **Evidence events** with a “Raw” view of each original log row, displayed as text and never as HTML.
- **Related events:** same source IP, user or host within ±60 minutes, excluding the evidence, for context.
- **Notes** with author and UTC time. Every status or verdict change adds an automatic note, which gives an audit trail.
- **Rule context:** limitations and false positives are shown next to the triage controls.

Suspected activity vs. confirmed incidents

Field	Values	Meaning
Status	New → In Progress → Closed	Position in the alert queue
Verdict	Suspected (default)	A rule matched; not yet verified. Every new alert starts here.
	Confirmed	Analyst verified malicious activity. **The API rejects this without a justification of at least 10 characters.**
	False Positive	Rule matched, but the activity was not malicious (useful for tuning)
	Benign	Real activity that is expected or authorized

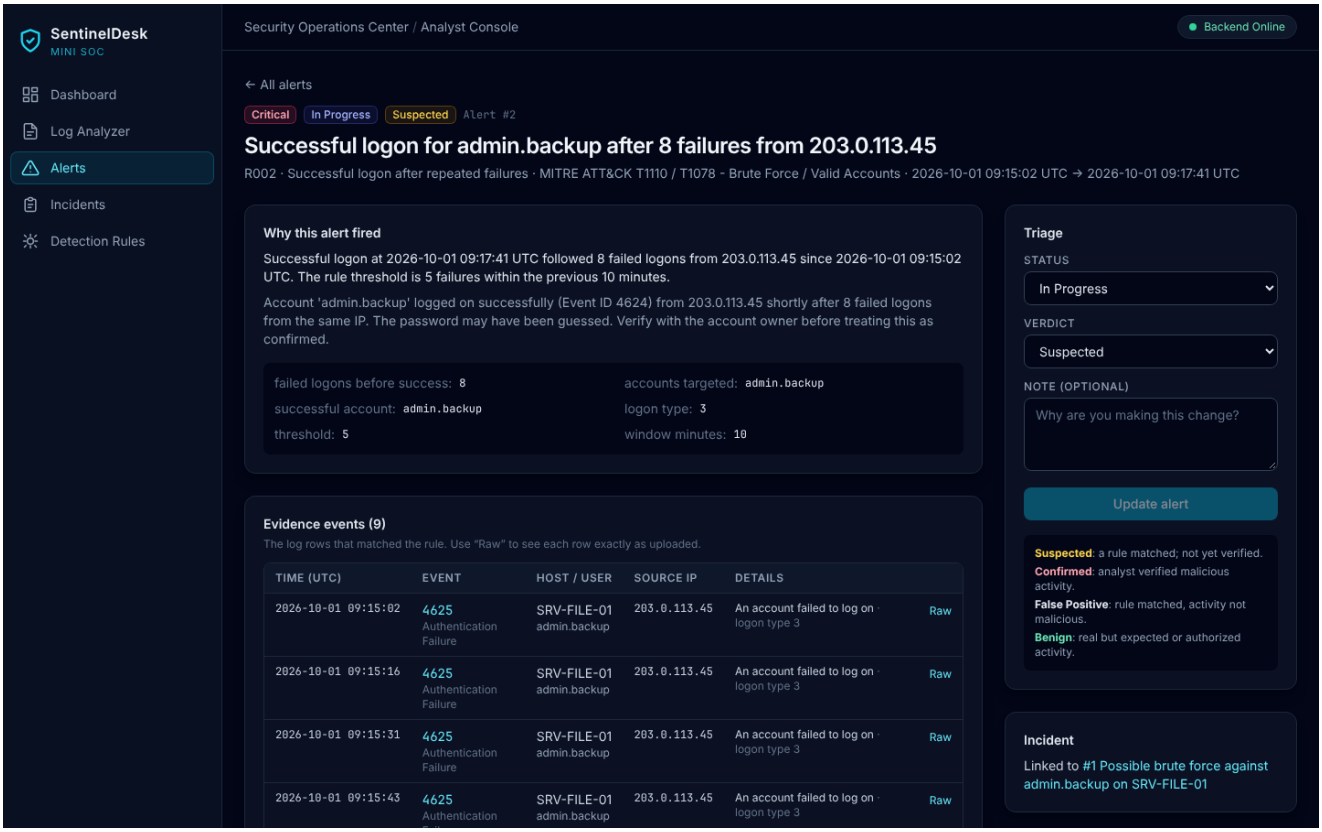


Figure 5 — Alert detail: explanation, evidence, triage controls and rule context

3.4 Incident Management

Status: Built and tested

Incidents are created from one or more alerts (pre-filled from the alert the analyst came from) or by hand. Each alert can belong to only one incident. Severity, priority, owner, status changes, linked alerts and notes are all written to an automatic timeline.

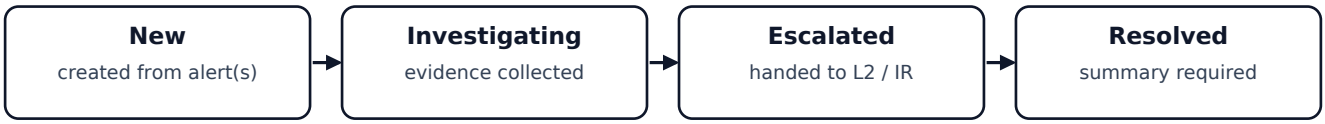


Figure 6 — Incident lifecycle. Escalated is optional; resolved incidents can be reopened.

Priority	Meaning (project convention)
P1	Active, high-impact threat; respond immediately
P2	Likely compromise or high-value asset affected; respond within hours
P3	Suspicious activity needing review; within one business day
P4	Low risk or informational; when capacity allows

- **Validated workflow:** invalid jumps (for example Investigating → New) are rejected, and Resolved needs a resolution summary.
- **Report export:** Markdown for people and JSON for tools. Both cover summary, linked alerts, evidence, analyst notes, timeline and resolution.
- **Honest assessment:** the report says how many alerts were confirmed, or states that the activity "remains suspected and should not be described as a confirmed attack".
- **Safe output:** log text in Markdown reports is escaped, so HTML in a username cannot run in a Markdown viewer.

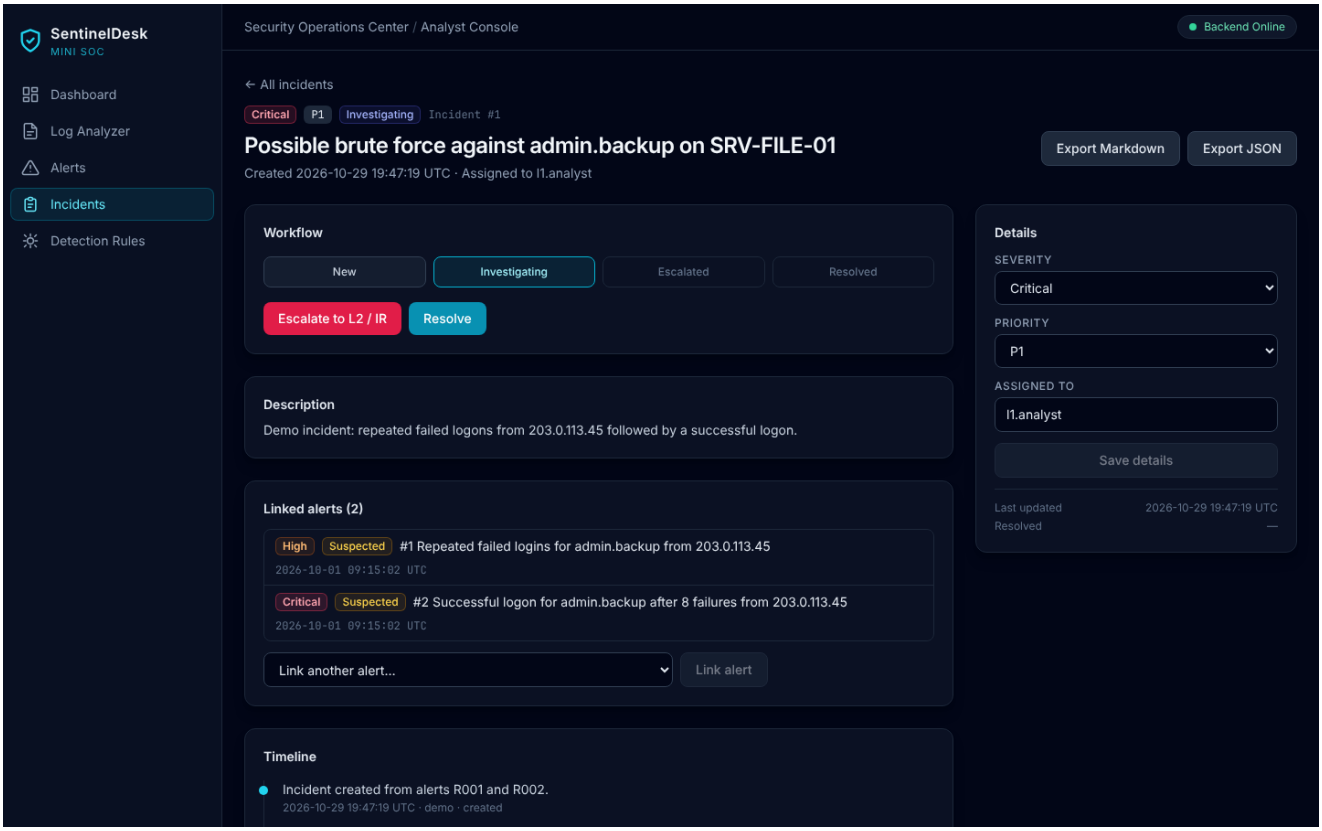


Figure 7 — Incident detail with workflow, linked alerts, timeline and export buttons

3.5 Detection Rules

Status: Built and tested

Rule **settings** (enabled, severity, threshold, window) live in the database and can be tuned on the Detection Rules page; new values apply to logs uploaded afterwards. Rule **logic** lives in `app/detection/rules/`, one file per rule.

ID	Rule	Default logic (configurable)	Severity	MITRE ATT&CK
R001	Repeated failed logins	5 or more × 4625 for the same user + source IP within 5 minutes; a long burst is one alert	**High**	T1110 Brute Force
R002	Successful logon after failures	4624 from an IP with 5 or more × 4625 in the previous 10 minutes; only the first success alerts	**Critical**	T1110 / T1078
R003	New user account created	Every 4720; names the creator when it is logged	**Medium**	T1136.001
R004	New service installed	Every 7045; raised to High if the path is in Temp, AppData or Users\Public (text match only)	**Medium**	T1543.003

Limitations and possible false positives

Rule	Limitations	Possible false positives
R001	Misses slow attacks below the threshold and password spraying (one try against many accounts).	Mistyped passwords; stale service-account credentials; many users behind one NAT IP.
R002	Cannot prove the same person made the failures and the success; stays Suspected until verified.	A user who forgot their password and then remembered it.
R003	Cannot see whether the creation was approved (no ticketing integration).	Normal onboarding of new employees.
R004	Only checks the path text; attackers can use normal-looking paths.	Software installs and updates, drivers, IT and security agents.

Test data. `suspicious_events.json` triggers all four rules (8 failures then a success from 203.0.113.45, a new account, a service in C:\\Users\\Public). `normal_events.json` is an ordinary day and triggers **no** alerts. `mixed_events.csv` triggers R001 (overnight RDP burst) and a Medium R004 (Program Files path). `invalid_events.json` demonstrates validation (2 valid, 6 rejected).

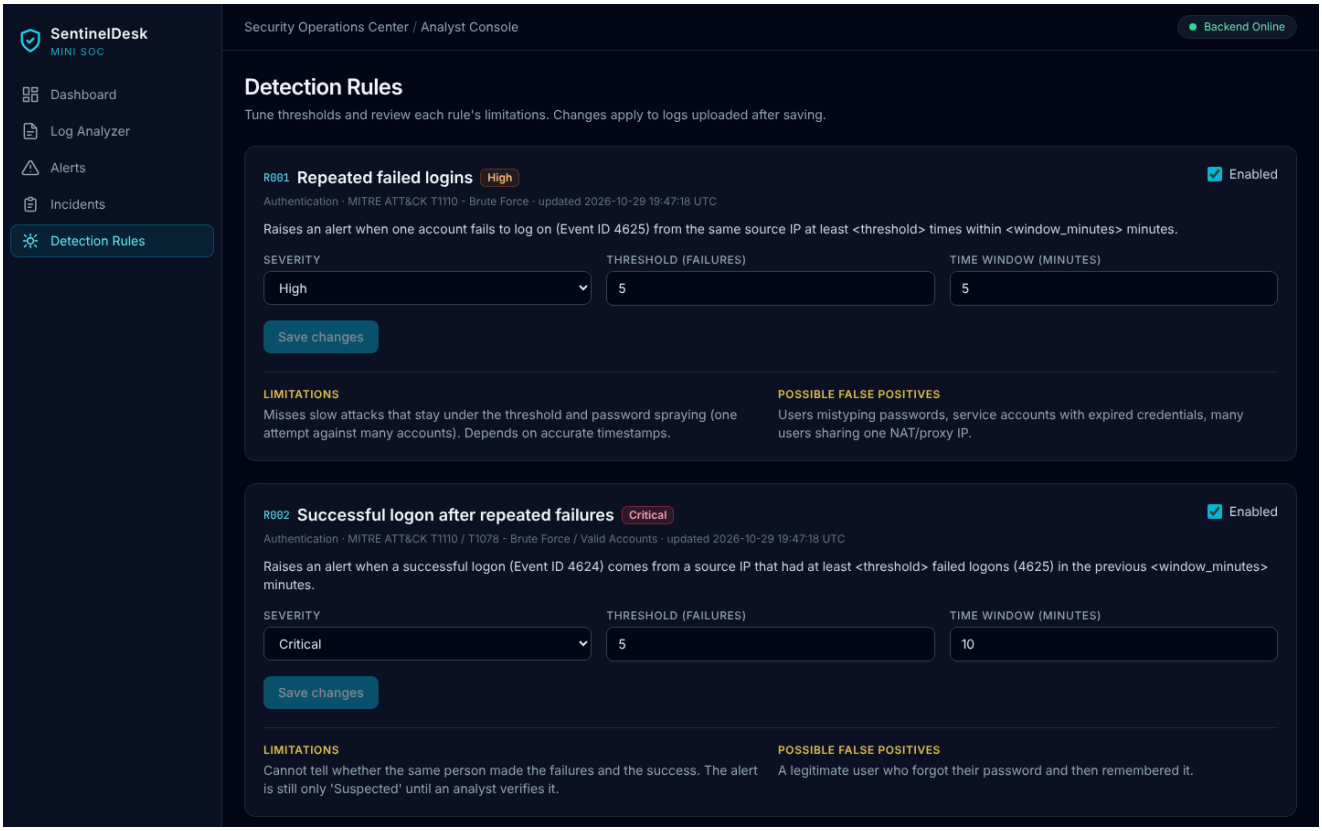


Figure 8 — Detection Rules page with tunable settings, limitations and false positives

3.6 Security requirements

Requirement	How SentinelDesk meets it	Status
Validate uploads; restrict formats and sizes	Extension allow-list, size and event limits from config, UTF-8 and binary checks, per-row validation	Built
Parameterized database queries	All access through the SQLAlchemy ORM with bound parameters; text search escapes % and _; tested with an injection attempt	Built
Never execute uploaded content	Parsed in memory with json/csv only; command lines and service paths are stored and shown as text	Built
Secrets in environment variables	pydantic-settings reads .env; .env is git-ignored and only .env.example is committed	Built
No real credentials, personal data or malware	Fictional users and hosts; reserved documentation IPs (198.51.100.x, 203.0.113.x) and private 10.x; no binaries	Built
No confirmed attack without evidence	Alerts default to Suspected; Confirmed requires a justification; reports state when nothing is confirmed	Built
Output escaping (XSS)	React escapes rendered text; raw rows shown as text; Markdown reports escape < > &	Built
Path tricks in filenames	Filenames reduced to a safe base name (../../x.json becomes x.json)	Built
Safe errors and CORS	Generic error messages (no stack traces); CORS allows only the frontend origin	Built

4. Database design

Table	Purpose	Key columns
log_uploads	One uploaded file and its validation summary	filename, file_type, size, total / accepted / rejected rows
security_events	One normalized log event	timestamp, event_id, event_category, hostname, username, source_ip, logon_type, service_name, service_path, raw_data
detection_rules	Tunable settings per rule	id, enabled, severity, threshold, window_minutes, mitre_technique, limitations, false_positives
alerts	A rule match awaiting triage	rule_id, severity, status, verdict, first/last seen, event_count, evidence, detection_explanation, incident_id
alert_events	Evidence links (many-to-many)	alert_id, event_id
alert_notes	Analyst and automatic notes	alert_id, author, note, created_at
incidents	A case grouping related alerts	title, severity, priority, status, assigned_to, resolution_summary, resolved_at
incident_timeline	Chronological incident record	incident_id, entry_type (created / status_change / updated / note / alert_linked), message, author

All times are stored in UTC, and SQLite foreign keys are enforced on every connection.

5. API reference

Method	Path	Purpose
GET	`/api/health`	Backend and database status
POST	`/api/logs/upload`	Upload a log; returns counts, row errors and generated alerts
GET	`/api/logs/uploads`	Upload history
GET	`/api/events[/{id}]`	Events with filters and paging / one event with its raw row
GET, PUT	`/api/rules[/{id}]`	List rules / tune enabled, severity, threshold, window
GET	`/api/alerts`	Alert queue with filters (severity, status, verdict, rule, in_incident, q)
GET, PATCH	`/api/alerts/{id}`	Alert detail / change status and verdict (with note)
GET, POST	`/api/alerts/{id}/related · /notes`	Related events / add a note
GET, POST	`/api/incidents`	List / create (with alert_ids)
GET, PATCH	`/api/incidents/{id}`	Detail / status, severity, priority, owner, resolution
POST	`/api/incidents/{id}/notes · /alerts`	Timeline note / link more alerts
GET	`/api/incidents/{id}/report`	Download report (format=markdown or json)
GET	`/api/dashboard/summary`	Dashboard numbers

Interactive documentation is generated automatically at http://localhost:8000/docs.

6. Testing and verification

Every test uses a fresh temporary database, so tests never touch real data. **149 tests pass**:

Test file	Tests	What it proves
test_log_parser.py	46	Every file- and row-level rejection, CSV quirks, UTC conversion, safe filenames, sample file counts
test_detection_rules.py	27	Each rule fires on suspicious data and stays quiet on normal data; threshold and window boundaries; tuning; disabled rules
test_api_alerts.py	19	Filters, evidence, related events, notes, status flow, Confirmed requires justification
test_api_logs.py	17	Uploads (201/413/415/422), nothing saved on failure, event filters, paging, SQL-injection attempt
test_api_incidents.py	13	Create and link, 409 double-link, transitions, resolution summary, timeline, reports, HTML escaping
test_models.py	8	Schema rules: defaults, cascades, foreign keys, idempotent rule seeding
test_api_rules.py	7	Rule listing and validation; tuned thresholds and disabled rules applied to the next upload
test_health.py	5	Health endpoint, 503 without leaking details, CORS allow / block
test_api_dashboard.py · test_report_service.py · test_demo_data.py	3 · 3 · 1	Dashboard counts, Markdown escaping, demo loader

Browser end-to-end check

During development, a Playwright script drove the real UI against the real API on an empty database. It checked the empty dashboard, uploaded the suspicious file (4 alerts), opened the Critical alert, added a note, confirmed that **Confirmed is blocked until a justification is written**, created an incident with two alerts, moved it through Investigating → Escalated → Resolved, downloaded the Markdown report (assessment: 1 of 2 alerts confirmed), and saw the dashboard update. The browser console showed no errors.

Build phases

Phase	Scope	Status
1	Foundation: FastAPI, configuration, database schema, health API, dark UI shell	**Complete**
2	Log ingestion and validation, Event ID catalog, synthetic sample data, events API	**Complete**
3	Detection engine, rules R001–R004, rules API	**Complete**
4	Log Analyzer and Detection Rules pages	**Complete**
5	Alert investigation API and pages	**Complete**
6	Incident management, timeline, report export	**Complete**
7	SOC dashboard and charts	**Complete**
8	Demo loader, documentation, screenshots, end-to-end verification	**Complete**

7. Running the project (Windows PowerShell)

Prerequisites: Python 3.12+, Node.js 18+ and Git.

Backend — PowerShell window 1

```
cd sentineldesk\backend
python -m venv .venv
.\.venv\Scripts\Activate.ps1
pip install -r requirements.txt
Copy-Item .env.example .env
pytest -v
python -m app.demo_data --reset
uvicorn app.main:app --reload --port 8000
```

If activation is blocked, run `Set-ExecutionPolicy -Scope Process -ExecutionPolicy Bypass` first. `--reset` deletes all data and then loads the samples (36 events, 6 alerts, 1 demo incident). API docs: http://localhost:8000/docs

Frontend — PowerShell window 2

```
cd sentineldesk\frontend
npm install
Copy-Item .env.example .env
npm run dev
```

Open http://localhost:5173. The header shows **Backend Online** when both parts are running.

Five-minute demo walkthrough

- **Dashboard:** stat cards, verdict strip, charts and the Table toggle.
- **Log Analyzer:** upload invalid_events.json (validation), then suspicious_events.json (4 alerts).
- **Alert R002:** why it fired, raw events, related events; add a note; try Confirmed (blocked until justified).
- **Incident:** create from the alert, move to Investigating → Escalated → Resolved, then export Markdown.
- **Detection Rules:** raise R001's threshold and explain its false positives.

8. Known limitations

- No login or roles. The analyst name is a free-text label, so authentication is needed before any real use.
- Detection runs per upload: there is no correlation across uploads, and re-uploading a file creates duplicate alerts.
- Changed thresholds apply to new uploads only; old logs are not re-scanned.
- SQLite and a single process are fine for a demo, not for high log volumes. Logs are uploaded by hand (no live collection).

9. Interview talking points

Q. What does SentinelDesk do?

It is a mini SOC platform. It ingests Windows-style security logs, runs four detection rules, raises alerts with evidence, and supports triage, incident handling and report export.

Q. Explain the brute-force rule.

It counts Event ID 4625 failures for the same user and source IP in a sliding window. At the threshold (5 in 5 minutes by default) it raises one High alert per burst. A 4624 success from that IP shortly after is more serious, so a separate rule raises it as Critical, but still only as Suspected.

Q. Why not mark detections as confirmed attacks?

A rule match is a signal, not proof. A user mistyping a password produces the same 4625 events as an attacker. Every alert starts as Suspected; the API only accepts Confirmed with a written justification, and reports say clearly when nothing has been confirmed.

Q. How do you reduce false positives?

Tune thresholds and windows per environment, allow-list known service accounts and NAT ranges, and use the False Positive and Benign verdicts to find noisy rules.

Q. Why is event 7045 important?

It records a new Windows service. Services start automatically, often as SYSTEM, so attackers use them for persistence. A service binary in Temp, AppData or Users\Public is unusual, so severity is raised.

Q. How is the application itself secured?

Uploads are restricted and validated row by row, content is parsed as data and never executed, queries are parameterized, output is escaped, secrets are in environment variables, CORS is restricted, and errors do not leak internals. Tests cover an SQL-injection attempt and HTML escaping in reports.

Q. What would you add next?

Password-spraying detection (one IP against many accounts), off-hours logon detection, Sigma rule import, authentication with roles, and live log ingestion such as Windows Event Forwarding.

Disclaimer. SentinelDesk is an educational portfolio project. It uses only synthetic data and is not intended to protect production systems.